

OPENING

"This is a live coding performance. The code runs in the browser in real time — every change I make appears instantly on screen. There is no pre-rendered video. Everything you see is generated by the computer reacting to music, right now."

THE STRUCTURE

"The whole piece is built in layers. Each layer is a function — a named block of code that does one visual job. I turn them on and off by commenting and uncommenting a single line. This is my instrument."

Show `draw()` and point to the commented lines.

THE AUDIO CONNECTION

"The computer is listening to the music through the microphone or a virtual audio cable. Every frame — 60 times per second — it analyzes the sound and gives us numbers. Low numbers for bass, high numbers for hi-hats, everything in between for mids. Those numbers go directly into the visuals."

Point to `fftEase` and `amp`.

FUNCTION BY FUNCTION

Uncomment each one and let it run for a few seconds before explaining:

THE GENERAL STRUCTURE

The code has two parts. At the top is `draw()` — this function runs 60 times per second, it's the engine. Below it are all the visual functions. `draw()` only calls the ones that are active — add `//` to comment a line and that layer turns off. It works exactly like a mixer.

The variable **t** is the internal clock. It doesn't

advance with real time — it advances with **amp**, the volume. `if (amp > thresh) t += amp * 0.003.`
Silence = t frozen = everything frozen.

KICK :

Imagine a stone dropped in water. A ring is born at the center, grows outward, fades before it reaches the edge, and resets. That's a phase loop — $(t * \text{speed} + i/n) \% 1$ — a number that goes from 0 to 1

and resets itself with $\% 1$. We run that loop **n** times simultaneously but each one starts at a different moment using **i/n**. The high frequencies `fftEase[0.75]` control how many rings exist and how fast they expand. **ellipse()** draws each one.

BASS

The screen is divided into 48 **columns**. Each column reads a different bin from the `fftEase` array in the low range — `map(i, 0, n, 0, fftEase.length * 0.10)`. If that frequency is loud, the column rises. `rect(x, height, width/n, -h)` draws from the bottom upward using a negative height. It's an equalizer drawn from scratch, 60 times per second.

WAVE

waveformEase is an array that p5live updates every frame with the **actual shape of the audio** — 1024 values between -1 and 1. We take that shape and draw it **waveCount** times using **beginShape()** and **curveVertex()**, each time at a different height calculated with **map(). curveVertex()** instead of **vertex()** makes the line smooth instead of jagged. The result is a curtain of visible sound.

KEYS:

A polygon is just points at equal angles around a circle — $\cos(i * 360/\text{sides}) * r$ for x, $\sin(i * 360/\text{sides}) * r$ for y. With sides = 6 we get a hexagon. We draw it 4 times inside the k loop, subtracting $k * 28$ from the radius each time to create concentric layers. **rotate(t * 0.15 + k * offset)** makes it spin and separates each layer. **push()** and **pop()** isolate each layer's rotation so they don't accumulate. The **mids fftEase[0.18]** make it expand.

HATS

Everything is controlled by $\text{power} = H / 255$ — the hi-hat volume normalized to 0–1. That single variable multiplies everything: n , sp , sz , al , vz . One knob controls it all. Each dot's position comes from $\text{noise}(i * 0.4, t * sp)$ — Perlin noise, the same algorithm used to simulate terrain and clouds. $i * 0.4$ gives each dot its own unique path, $t * sp$ advances it through time. When $sp = 0$ in silence, the time coordinate in the noise freezes and the dots stop moving.

THE WORDS

These are song lyrics. Their position comes from $\text{noise}(t * 0.006 + i * 0.4, i * 0.3)$ — each word has its own seed $i * 0.4$ so it follows its own path without syncing with the others. $\text{amp} * 0.6$ controls opacity — they appear with volume and vanish in silence. $\text{text}(\text{words}[i \% \text{words.length}], x, y)$ cycles through the words array using $\%$. All four functions are identical in structure — only the $\text{words}[]$ array, the color in $\text{fill}()$, and the $\text{textSize}()$ change.

"The functions are completely independent. They don't talk to each other. I can have one running, five running, or none. Each one is a layer I add or remove in real time — like mixing tracks on a DJ board, but instead of audio I'm mixing visuals."

"There's also one shared variable at the top — t. It's a timer that only moves forward when there's audio. When the music stops, t freezes, and every function that uses it freezes too. Silence is not empty — it's a held frame."

Point to:

```
if (amp > thresh) t += amp * 0.003
```

"This one line is what makes the whole piece reactive. Louder audio = t moves faster = animations accelerate. It's the heartbeat of the system."